Node.js — Lecture 1

Detailed Study Notes

Introduction • V8 Engine • Installation • Running JavaScript • JavaScript Basics

Prepared from the supplied lecture transcript

These notes organize the lecture into a study-friendly reference, preserve the instructor's key explanations and examples, and clarify obvious speech-to-text errors in technical terms.

Topics covered

- What Node.js is and why it is useful for backend development
- Client-side vs server-side code, with shopping-site and restaurant analogies
- V8 JavaScript engine and the path from source code to execution
- Installing Node.js, npm, VS Code, and Postman
- Running JavaScript in an online editor, browser console, Node terminal/REPL, and VS Code
- Variables, constants, data types, arrays, conditions, loops, objects, array filtering, and user input
- Practice tasks and the instructor's learning advice

1. Learning goals and prerequisites

The lecture begins with the instructor explaining why Node.js is worth learning: backend skills can strengthen a developer's résumé and help with interviews and projects. The goal is to build enough confidence to start solving basic Node.js problems and creating simple backend functionality.

Prerequisites

- A computer: laptop or desktop.
- Time and dedication to practise while watching the lecture.
- JavaScript knowledge is helpful, but the instructor says JavaScript basics will also be introduced.
- A hands-on approach: pause the video, type the code, run it, and experiment rather than only watching.

Instructor's study advice: do not postpone all practice until the end. Follow along, try the commands yourself, investigate errors, and make your own notes after the lecture. The instructor repeatedly encourages learners to Google errors and explore settings independently.

2. What is JavaScript, and what is Node.js?

JavaScript

JavaScript is a programming language, like C++, Python, and Java. It is commonly used in web pages to add interactivity—for example, responding to button clicks, changing UI content or colours, and displaying pop-ups.

Node.js

Node.js is a JavaScript runtime environment that lets JavaScript run outside the browser, including on a server. It uses the V8 JavaScript engine and provides runtime capabilities useful for building backend applications.

- Before Node.js became widely used, JavaScript was mainly associated with browser/front-end work.
- With Node.js, developers can use JavaScript for both front-end and back-end development.
- A common advantage is that a developer can use one language across more of the application, although the browser and server still have different responsibilities.

Important distinction: JavaScript is the language; Node.js is a runtime for executing JavaScript outside the browser. Node.js is not a replacement for JavaScript—it provides an environment in which JavaScript can run on the server.

3. Client side vs server side

Client side (front end)

- The part of an application that runs in the user's browser and presents the interface.
- Examples: search bars, buttons, product cards, and displaying the results returned by a server.

Server side (back end)

- The part that handles application logic and requests behind the scenes.
- It may validate a request, interact with a database, process a purchase, update inventory, and send a response to the client.

Shopping-site example from the lecture

Imagine searching for a shirt on an online shopping site:

- You type "shirt" into a search bar. The search interface is part of the client side.
- The browser sends a request to the server asking for matching products.

- The server may query the database and retrieve matching shirts.
- The server sends product information back in a response.
- The client-side code displays the products on the screen.
- If you click “Buy Now,” the client sends another request. The server handles the purchase workflow, payment-related processing, and inventory update.

The instructor gives the example of stock changing from 60 shirts to 59 after one purchase. The key idea is that a visible button click is not itself the purchase: server-side processing must perform the relevant operations.

Restaurant analogy

The dining area and the food presented to the customer represent the client side. The kitchen, where the food is prepared, represents the server side. A customer sees the finished dish, not all the work happening in the kitchen. Similarly, a website user sees the interface while backend logic runs behind it.

Node.js lets a developer use JavaScript skills to build that backend as well as browser-side functionality.

4. V8 JavaScript engine

The instructor highlights V8 as an important interview topic: what technology powers Node.js?

- **V8** is Google’s open-source JavaScript engine, originally developed for running JavaScript efficiently in Chrome.
- Node.js uses V8 to execute JavaScript code.
- The engine processes JavaScript so the computer can execute it. Modern JavaScript engines use compilation and runtime optimisation techniques; it is an oversimplification to say every line is merely translated directly into machine code in one simple step.

Conceptual execution flow

```
JavaScript source code → V8 engine → executable machine instructions → CPU executes instructions
```

For example, a developer writes human-readable JavaScript such as `let a = 5`. The engine handles execution, and the CPU performs the underlying operations. The lecture uses this simplified flow to explain why an engine is needed.

Remember: V8 is the JavaScript engine; Node.js is the runtime environment built around V8 with additional APIs and capabilities for server-side work.

5. Tools to install

5.1 Node.js and npm

Install Node.js from the official Node.js website. The lecture recommends the LTS (Long-Term Support) release for most learners. Follow the installer for your operating system and complete the installation.

npm is the Node.js package manager. It is normally installed along with Node.js and is used to install and manage JavaScript packages/dependencies.

Verify installation by opening Command Prompt/Terminal and running:

```
node -v

npm -v
```

Each command should print a version number. The lecture shows an example version, but versions change over time, so your number may be different.

If installation fails

- Read the error message instead of repeatedly reinstalling without checking.

- Search the exact error message and your operating system; follow a reliable, current installation guide.
- Check that the terminal was reopened after installation and that Node.js is available on PATH.
- Use the version commands above to confirm whether the tools are available.

5.2 Visual Studio Code (VS Code)

VS Code is the code editor used in the lecture. Install the version for your operating system (Windows, macOS, or Linux). The instructor recommends becoming familiar with the editor because it is widely used by developers.

- Create and open project folders and files.
- Save code and use the integrated terminal.
- Learn where extensions and editor settings are located.
- Optional productivity setting: enable Auto Save in VS Code settings so edits are saved automatically.

5.3 Postman

Postman is a tool for testing APIs. It becomes useful when building backend endpoints and sending requests with different methods, headers, and payloads. The instructor recommends installing it in advance so it is ready for later API lessons.

At this stage, the lecture introduces Postman; it does not yet teach a complete API-testing workflow.

6. Four ways to run JavaScript

The lecture demonstrates four approaches. Use the online editor for quick experiments, the browser console for browser-side JavaScript, Node's REPL for quick terminal tests, and VS Code for normal file-based development.

6.1 Online JavaScript editor

Search for an online JavaScript editor/compiler, type code, and press Run. The instructor demonstrates adding two numbers.

```
let a = 5; let b = 6; let answer = a + b; console.log(answer); // 11
```

`console.log()` prints a value to the console. It is comparable in purpose to output statements such as `cout` in C++ or `print` in Python.

6.2 Browser developer tools / console

- Open a web page in a browser.
- Open Developer Tools (often by right-clicking and selecting Inspect, or using the browser's developer-tools shortcut).
- Select the Console tab.
- Type JavaScript statements and press Enter.

```
let a = 5; let b = 6; let answer = a + b; console.log(answer); // 11
```

The console is useful for quick experiments and for inspecting browser-side behaviour. A browser also provides browser-specific APIs, such as `prompt()`.

6.3 Node.js REPL in the terminal

REPL stands for Read–Eval–Print Loop. It lets you enter JavaScript expressions interactively.

```
node
```

After the Node prompt opens, enter statements such as:

```
let a = 5; let b = 6; let answer = a + b; answer
```

Typing a variable name in the REPL can display its value. To exit, press Ctrl+C twice or use `.exit` (depending on the terminal/REPL state).

6.4 Run a JavaScript file using Node.js and VS Code

Create a project folder, open it in VS Code, and create a file such as `main.js`. JavaScript files use the `.js` extension.

```
let a = 5; let b = 6; let answer = a + b; console.log("Answer is", answer);
```

Save the file, open a terminal in the folder containing it, and run:

```
node main.js
```

Expected output:

```
Answer is 11
```

If the code is changed but the file is not saved, Node may execute the previous saved version. Save with Ctrl+S on Windows/Linux or Cmd+S on macOS. Auto Save can reduce this issue.

7. JavaScript basics introduced in the lecture

7.1 Variables: let and var

Variables store values. The lecture introduces variables by storing two numbers and adding them.

```
let a = 5; let b = 6; let answer = a + b; console.log(answer); // 11
```

`let` declares a block-scoped variable that can be reassigned. `var` is an older declaration keyword with function scope and some different redeclaration/hoisting behaviour. For modern beginner code, prefer `const` by default and `let` when reassignment is needed.

7.2 Constants: const

A `const` declaration must be given an initial value and cannot be reassigned later.

```
const name = "Prince"; console.log(name);
```

This is invalid because it attempts to reassign a constant:

```
const name = "Prince"; name = "Ajay"; // TypeError
```

Important nuance: `const` prevents reassignment of the binding; it does not automatically make the contents of an object or array immutable.

7.3 Data types and typeof

JavaScript variables are dynamically typed: you do not declare a variable as an integer or string in the same way as in C++. A variable can hold values of different types over its lifetime if declared with `let` or `var`.

```
const name = "Prince"; const answer = 5 + 6; console.log(typeof name); // "string"
console.log(typeof answer); // "number"
```

`typeof` is an operator that reports the type category of a value. Common results include `"string"`, `"number"`, `"boolean"`, `"undefined"`, `"object"`, and `"function"`. Note: `typeof null` returns `"object"` for historical reasons.

7.4 Arrays

An array stores an ordered collection of values. The lecture uses a `cars` array and demonstrates printing the whole array, adding an item, and reading items by index.

```
const cars = ["Tesla", "Audi", "BMW"]; console.log(cars); cars.push("Toyota");
console.log(cars); console.log(cars[0]); // first item console.log(cars[2]); // third item
```

- Array indexes start at 0, not 1.
- `cars[0]` is the first element; `cars[1]` is the second; `cars[2]` is the third.

- `.push(value)` adds an item to the end of an array.

The lecture also notes that arrays can contain values without a fixed, declared element type. JavaScript permits mixed types, although keeping arrays consistent is often clearer.

7.5 Conditional statements: if / else

An `if` statement runs a block when its condition is true. The lecture illustrates checking whether a value is below a threshold.

```
const age = 10; if (age < 12) { console.log("Not allowed"); } else { console.log("Allowed"); }
```

The exact message depends on the condition and the example being tested. Practise changing the value (for example, 10, 12, and 16) and predict the output before running the code.

7.6 Loops

A loop repeats a block of code. The lecture demonstrates printing numbers from 1 through a count (shown as 10).

```
const count = 10; for (let i = 1; i <= count; i++) { console.log(i); }
```

- Initialise `i` at 1.
- Continue while `i <= count`.
- Increase `i` by 1 after each iteration.

Because the condition is `i <= count`, this example prints 1, 2, 3, …, 10.

8. Objects and accessing properties

An object groups related information as key–value pairs. The instructor uses a person-like object with a name, a student flag, and a hobbies array.

```
const person = { name: "John", isStudent: true, hobbies: ["reading", "gaming", "running"] };
console.log(person); console.log(person.name); console.log(person.hobbies);
```

- `person` is the object variable.
- `name`, `isStudent`, and `hobbies` are property keys.
- `person.name` accesses the value stored under the `name` property.
- `person.hobbies` returns the array stored in the `hobbies` property.

Objects are heavily used in JavaScript to represent structured data. The dot notation pattern is `objectName.propertyName`. Bracket notation, such as `person["name"]`, is another way to access a property.

The instructor flags objects as an important topic that will appear frequently in later JavaScript/Node.js work.

9. Array filter() and callback functions

The lecture introduces `filter()` using an array of numbers and the goal of keeping values that meet a condition (in the example, values less than or equal to 18).

```
const ages = [32, 16, 18, 40]; const result = ages.filter(function (age) { return age <= 18;
}); console.log(result); // [16, 18]
```

How it works:

- `.filter()` visits each element in the array.
- For each element, it calls the callback function and passes that element as the parameter (`age` in this example).
- The callback must return a truthy/falsy result. Elements for which it returns true are included in the new array.
- `filter()` returns a new array; it does not automatically print it. Use `console.log(result)` to see the result.

The instructor initially runs the filter operation without printing the result and explains that nothing visible appears until the result is logged. This is a useful debugging lesson: calculating a value and displaying it are separate actions.

Same example using an arrow function

```
const result = ages.filter(age => age <= 18); console.log(result); // [16, 18]
```

A callback is a function passed to another function to be called by it. `filter()` is one common place where callbacks are used.

10. Taking user input: prompt and prompt-sync

Browser prompt()

In a browser, `prompt()` opens a dialog asking the user for input.

```
const age = prompt("Please enter your age:"); console.log(age);
```

Browser `prompt()` generally returns the entered value as a string, or `null` if the user cancels. Convert it to a number when doing numeric comparisons, for example `Number(age)`.

Why prompt() may fail in Node.js

The instructor demonstrates that `prompt()` is not defined in a standard Node.js environment. `prompt()` is a browser API, not a built-in Node.js global.

Using the prompt-sync package

The lecture recommends installing a package from npm to request terminal input synchronously.

```
npm install prompt-sync
```

Example usage:

```
const prompt = require("prompt-sync")(); const ageInput = prompt("Please enter your age: ");
const age = Number(ageInput); if (age >= 18) { console.log("Adult"); } else {
console.log("Under 18"); }
```

Install the package in the project folder where the script will run. `require("prompt-sync")()` loads the package and creates a prompt function. Input arrives as text, so convert it with `Number()` when you need a numeric value.

Module note: this example uses CommonJS `require`, which works in a typical Node.js project configured for CommonJS. Projects configured as ES modules may use a different import approach.

11. Practical workflow and terminal tips

- Use `node -v` and `npm -v` to check the installation.
- Use `node` to open the interactive REPL.
- Use `node main.js` to execute a saved JavaScript file.
- Use `ls` in macOS/Linux terminals to list files; Windows Command Prompt commonly uses `dir`.
- Use the terminal in the project folder so Node can find the file and any installed dependencies.
- In the REPL, use the up-arrow key to recall previous commands.
- Use `clear` in many macOS/Linux terminals to clear the terminal display; in Windows Command Prompt, use `cls`. Clearing the screen does not delete your files or undo code.
- Save changes before running a file. If output looks unchanged, check whether the file was saved and whether you are running the correct file.
- Experiment with arithmetic operators such as `+`, `-`, `*`, and `/`, and predict the output before running the code.

12. Key terms — quick revision

Term	Meaning
JavaScript	Programming language used in browsers and, with a runtime such as Node.js, on servers.
Node.js	Runtime environment for executing JavaScript outside the browser.
V8	Google's JavaScript engine used by Node.js.
Client side	Browser/UI side that presents content and sends requests.
Server side	Backend side that processes requests and may access a database.
npm	Package manager used to install and manage JavaScript packages.
VS Code	Code editor used to create and run project files.
Postman	Tool for sending requests to and testing APIs.
REPL	Interactive Read–Eval–Print Loop for running expressions.
console.log()	Prints values to the console.
const	Declares a binding that cannot be reassigned.
typeof	Operator that reports the type category of a value.
Array	Ordered collection; indexes begin at zero.
Object	Collection of key–value properties.
filter()	Creates a new array containing elements that pass a test.
Callback	Function passed to another function for it to call.

13. Practice tasks from this lecture

The instructor says the accompanying lesson material contains five problem statements and solutions. The advice is to attempt the problems independently first, run the code, and only then consult the provided solutions. The exact five statements are not included in the supplied transcript, so they cannot be reproduced reliably here.

Practice these skills

- Write a program that adds two numbers and prints the result.
- Create variables with `let` and `const`; predict which assignments are valid.
- Use `typeof` to inspect a string, number, and boolean.
- Create an array, print it, add an element with `push()`, and access items by index.
- Write an `if/else` condition and test values on both sides of the threshold.
- Use a `for` loop to print numbers from 1 to 10.
- Create a person object and print one property and an array property.
- Use `filter()` to keep numbers below or equal to a threshold; print the returned array.
- Install `prompt-sync`, take a value from the terminal, convert it to a number, and use it in a condition.
- Save a file as `main.js` and run it using `node main.js`.

14. Common mistakes to avoid

- Confusing Node.js with JavaScript: one is a runtime; the other is the language.

- Assuming a browser API such as `prompt()` is automatically available in Node.js.
- Expecting `filter()` to print by itself: store or directly log the returned array.
- Forgetting that array indexes start at zero.
- Trying to reassign a `const` variable.
- Running a file from the wrong folder or forgetting to save it.
- Expecting numeric input to be a number: prompt input is usually text and may need conversion.
- Copying code without testing it. Type, run, change values, and observe the result.

15. Final recap

This lecture establishes the foundation for the Node.js course: what Node.js is, how it relates to JavaScript and V8, why client-side and server-side responsibilities differ, and which tools are needed. It then introduces ways to execute JavaScript and revises core language features—variables, constants, types, arrays, conditions, loops, objects, callbacks, `filter()`, and user input. The central learning method is practical: install the tools, follow along, run every example, investigate errors, and attempt the assigned problems before checking solutions.

Source note: These notes are based on the transcript supplied by the user. Speech-to-text spellings were normalized for technical terms (for example, Node.js, V8, VS Code, npm, `console.log`, `filter()`, and `prompt-sync`). Explanatory clarifications are included where needed to make the concepts technically accurate.